



Singapore University of Technology and Design

50.043 Database Systems

Group 2

SimpleDB - Lab3

<i>Authors</i>	<i>Student ID</i>
Jone Chong Jin	1006338
Sarang Nambiar	1006181
Loh Jianyang John	1006360

13/04/2025

Implementation Details:

1. Explain any design decisions you made. For deadlock handling (detection and resolution), there are several alternatives. List the pros and cons of your approach.

BufferPool.java

For deadlock detection, we had partitioned the lock acquirers and holders based on the type of lock they are trying to acquire or hold. This gave us a more fine-grained control on which transactions are trying to acquire/hold a Shared/Exclusive lock (SL/EL). This improves the robustness of our `if` conditions to check for potential deadlocks.

In dependency cycle detection for deadlocks, we used Depth First Algorithm (DFS) with caching (visited set) for a snappy and straightforward implementation. A `HashMap` of `TransactionId` to a set of `TransactionIds` it is dependent on was used as the adjacency list for graph traversal purposes.

Cycle Detection + Thread Abort Approach:

Pros:

- Accurate deadlock detection unlike timeout based strategies since the transaction just might be slow.
- It is quite fast, with an $O(n)$ time complexity.

Cons:

- Must update the dependency graph and run cycle checks every time a transaction tries to acquire a lock, which could pose significant overhead when the number of transactions increases.
- If the graph gets too complex and deep, DFS could consume significant amounts of stack memory.

2. The lab assumes locking at page level. How would your code/design change if you were to adopt tuple-level locking?

Exercise 1 and 2:

At a tuple-level locking, this would require us to make the following changes:

Firstly the `PageLock` will now be changed into a `TupleLock` where we now track transactions that already hold or are acquiring Shared and Exclusive locks on tuples (`recordIds`) instead of pages (`pageIds`). Similarly, the `LockManager` will now track `recordIds` instead of `PageIds` in its HashMaps and it will now use the `TupleLock`. Both APIs now work together to track `TransactionIds` on a tuple level. Finally, locks (`PageLock`) were previously acquired in the `getPage()` method of the `BufferPool`. But now, the locks (`TupleLock`) will be acquired in the `BufferPool` methods `insertTuple()` and `deleteTuple()` as we are dealing with tuple-level locking.

Exercise 3:

The eviction logic would need to consider whether any tuple within a page is locked, rather than just checking the page's dirty status. A clean page might still have locked tuples, which should not be evicted. This would require the eviction policy to check the lock status of all tuples within each page.

Exercise 4:

Commit/abort logic would need to change. Instead of flushing or discarding entire pages, we would need to track and handle dirty tuples individually. On commit, only the tuples modified by the transaction would be flushed.

3. Discuss and justify any changes you made to the API.

No changes were made to the original API for the implementation of this task.

4. Describe any missing or incomplete elements of your code.

For exercise 5, the current way of handling deadlocks is through throwing the `ThreadAbortException`. This could be replaced with a more elegant way of handling deadlocks.